ASSESSMENT TOOL 2 – PSEUDOCODE WRITING TASK (ALGORITHM DESIGN)

Q1. DMA-Based Data Transfer Algorithm (I/O Device to Memory, Minimizing CPU Involvement)

1.1 Problem Understanding

Inputs: source I/O device address, destination memory buffer address, transfer length. Output: data transferred to memory with completion signaled via interrupt. Constraint: CPU must only configure and be notified — it must not participate in the actual data movement.

1.2 Pseudocode

```
BEGIN DMA_Transfer(device_id, src_addr, dest_addr, length)
    // Step 1: CPU configures the DMA controller
    DMA_Controller.SET(source = src_addr)
    DMA_Controller.SET(destination = dest_addr)
    DMA_Controller.SET(transfer_length = length)
    DMA_Controller.SET(mode = BLOCK_TRANSFER)
    DMA_Controller.SET(interrupt_on_completion = TRUE)

    // Step 2: CPU issues a single start command and is released
    DMA_Controller.START()
    CPU.RETURN_TO(other_tasks)   // CPU is now free

    // Step 3: DMA controller autonomously moves data (hardware, no CPU
cycles)
    WHILE (bytes_transferred < length) DO
        DMA_Controller.TRANSFER_BLOCK(device_id)
        bytes_transferred = bytes_transferred + block_size
        IF (device_status == ERROR) THEN
            DMA_Controller.RAISE_INTERRUPT(type = ERROR)
            ABORT_TRANSFER()
        END IF
    END WHILE

    // Step 4: On completion, hardware raises an interrupt
    DMA_Controller.RAISE_INTERRUPT(type = TRANSFER_COMPLETE)
END

// Interrupt Service Routine, executed only once per transfer
ON INTERRUPT(type = TRANSFER_COMPLETE) DO
    ACKNOWLEDGE_INTERRUPT()
    MARK_BUFFER_READY(dest_addr)
    NOTIFY_WAITING_PROCESS()
END
```

1.3 Explanation

- The CPU's role is limited to a one-time setup (source, destination, length) and a single START command — the WHILE loop that performs the actual transfer runs entirely inside the DMA controller's hardware, not on the CPU.
- Interrupts are used only for exception handling (errors) and final completion notification, so the CPU is invoked at most twice per transfer regardless of the transfer size, satisfying the 'minimizing CPU involvement' requirement.
- This cycle-stealing/burst-mode DMA pattern is the same one used by real NVMe, disk, and network controllers to move large blocks of data without CPU intervention.

Q2. Interrupt Handling System with Vectored, Prioritized Dispatch

2.1 Problem Understanding

Inputs: interrupt requests from multiple devices, each with an assigned priority and a vector (address of its ISR). Output: correct ISR executed for the highest-priority pending interrupt, with lower-priority interrupts queued or preempted appropriately.

2.2 Pseudocode

```
STRUCT InterruptVectorTable[device_id] = ISR_Address
STRUCT DevicePriority[device_id] = priority_level
QUEUE pending_interrupts = EMPTY
current_priority = NONE

ON HARDWARE_INTERRUPT(device_id) DO
    new_priority = DevicePriority[device_id]

    IF (current_priority == NONE OR new_priority > current_priority) THEN
        // Higher priority: preempt current ISR (nested interrupt)
        IF (current_priority != NONE) THEN
            SAVE_CONTEXT(current_ISR_state)
            PUSH pending_interrupts, current_interrupt
        END IF
        DISPATCH_ISR(device_id, new_priority)
    ELSE
        // Lower/equal priority: queue for later service
        ENQUEUE(pending_interrupts, device_id, new_priority)
        SORT(pending_interrupts BY priority DESCENDING)
    END IF
END

FUNCTION DISPATCH_ISR(device_id, priority)
```

```
    current_priority = priority
    isr_address = InterruptVectorTable[device_id]   // direct vectored
lookup
    JUMP TO isr_address                              // no polling
required
    EXECUTE_ISR(device_id)
    ACKNOWLEDGE_INTERRUPT(device_id)

    IF (pending_interrupts NOT EMPTY) THEN
        next = DEQUEUE_HIGHEST(pending_interrupts)
        RESTORE_CONTEXT(next)
        DISPATCH_ISR(next.device_id, next.priority)
    ELSE
        current_priority = NONE
        RETURN_FROM_INTERRUPT()
    END IF
END FUNCTION
```

2.3 Explanation

- Each device's ISR address is fetched directly from InterruptVectorTable using its device_id — this is the defining feature of vectored interrupts, avoiding any sequential polling to identify the source.
- The priority check enables true nesting: a higher-priority interrupt preempts a lower-priority ISR in progress, while equal/lower-priority interrupts are queued and sorted, ensuring the most urgent device is always serviced first.
- After each ISR completes, the algorithm automatically dispatches the next highest-priority pending interrupt (tail-chaining), minimizing dispatch latency between successive interrupts.

Q3. Dynamic Selection Between Memory-Mapped I/O and I/O-Mapped I/O

3.1 Problem Understanding

Inputs: a device's required latency threshold and bandwidth requirement. Output: a decision (MEMORY_MAPPED or IO_MAPPED) and configuration of the device's access mode accordingly. The decision should favor low latency/high bandwidth needs toward MMIO, and simple, low-bandwidth, isolated register access toward I/O-mapped.

3.2 Pseudocode

```
CONST LATENCY_THRESHOLD = 100   // nanoseconds
CONST BANDWIDTH_THRESHOLD = 50  // MB/s
```

```
FUNCTION SELECT_IO_MODE(device)
    required_latency  = device.max_acceptable_latency
    required_bandwidth = device.expected_bandwidth
    address_space_available = SYSTEM.CHECK_FREE_MEMORY_SPACE()

    IF (required_latency < LATENCY_THRESHOLD
        OR required_bandwidth > BANDWIDTH_THRESHOLD) THEN

        IF (address_space_available == TRUE) THEN
            mode = MEMORY_MAPPED
        ELSE
            // Fall back if no memory address space can be reserved
            mode = IO_MAPPED
            LOG_WARNING("Insufficient address space for MMIO")
        END IF

    ELSE
        // Low bandwidth, latency-tolerant, simple control device
        mode = IO_MAPPED
    END IF

    device.access_mode = mode
    CONFIGURE_DEVICE(device, mode)
    RETURN mode
END FUNCTION

FUNCTION CONFIGURE_DEVICE(device, mode)
    IF (mode == MEMORY_MAPPED) THEN
        MAP_TO_ADDRESS_SPACE(device, base_addr = ALLOCATE_MMIO_REGION())
        ENABLE_LOAD_STORE_ACCESS(device)
    ELSE
        ASSIGN_IO_PORT(device, port = ALLOCATE_IO_PORT())
        ENABLE_IN_OUT_INSTRUCTIONS(device)
    END IF
END FUNCTION
```

3.3 Explanation

- The decision logic checks both latency and bandwidth requirements: devices needing sub-100 ns response or more than 50 MB/s throughput are routed to memory-mapped I/O, since MMIO allows use of the CPU's fast, pipelined load/store path.
- A fallback to I/O-mapped access is included when insufficient memory address space is available for MMIO, ensuring the algorithm degrades gracefully rather than failing.

- Simple, low-bandwidth, latency-tolerant devices (e.g., basic status/control registers) default to I/O-mapped access, preserving memory address space for higher-need devices — reflecting real system design trade-offs.

Q4. Bus Arbitration Algorithm for PCIe, USB, and Thunderbolt Devices

4.1 Problem Understanding

Inputs: a set of devices, each tagged with its bus type (PCIe, USB, Thunderbolt) and a bandwidth/priority requirement. Output: an ordered grant of bus access ensuring fairness across devices and no single device or bus type monopolizes shared bandwidth.

4.2 Pseudocode

```
STRUCT Device { id, bus_type, priority, pending_request, last_service_time
}
LIST all_devices
CONST TIME_QUANTUM = fixed_slot_duration

FUNCTION BUS_ARBITRATE()
    WHILE (SYSTEM_RUNNING) DO
        requesting = FILTER(all_devices WHERE pending_request == TRUE)

        IF (requesting IS EMPTY) THEN
            WAIT_FOR_NEXT_REQUEST()
            CONTINUE
        END IF

        // Step 1: group by bus type (each bus arbitrated independently)
        FOR EACH bus IN {PCIe, USB, Thunderbolt} DO
            bus_devices = FILTER(requesting WHERE bus_type == bus)
            IF (bus_devices IS EMPTY) THEN CONTINUE

            // Step 2: sort by priority, then by longest-waiting
(fairness)
            SORT(bus_devices BY priority DESCENDING,
                               last_service_time ASCENDING)

            grantee = bus_devices[0]

            // Step 3: grant bus for one bounded time quantum
            GRANT_BUS(grantee.id, bus, duration = TIME_QUANTUM)
            TRANSFER_DATA(grantee.id)
            grantee.last_service_time = CURRENT_TIME()
            grantee.pending_request = FALSE
```

```
        END FOR
    END WHILE
END FUNCTION
```

4.3 Explanation

- Devices are first grouped by their physical bus (PCIe, USB, Thunderbolt) since each bus has independent bandwidth and must be arbitrated separately, mirroring real hardware where these are physically distinct channels.
- Within each bus group, devices are sorted by priority first and then by how long they have been waiting (last_service_time), which implements an aging/fairness mechanism so a low-priority device is never starved indefinitely.
- Granting bus access for a bounded TIME_QUANTUM rather than until completion ensures no single device — even a high-priority one — can monopolize the bus, satisfying fair arbitration across all connected peripherals.

Q5. Hybrid I/O Communication Mechanism (Polling for Low-Speed, DMA + Interrupts for High-Speed)

5.1 Problem Understanding

Inputs: a set of I/O devices, each classified by speed/throughput. Output: low-speed devices are serviced via polling (simple, low overhead for infrequent events), while high-speed devices are serviced via DMA with interrupt-based completion notification (to avoid CPU bottlenecking on high-throughput transfers).

5.2 Pseudocode

```
CONST SPEED_THRESHOLD = 10   // MB/s, boundary between low- and high-speed
LIST low_speed_devices, high_speed_devices

FUNCTION CLASSIFY_DEVICES(device_list)
    FOR EACH device IN device_list DO
        IF (device.throughput < SPEED_THRESHOLD) THEN
            ADD device TO low_speed_devices
        ELSE
            ADD device TO high_speed_devices
            CONFIGURE_DMA(device)
            ENABLE_INTERRUPT(device, on = TRANSFER_COMPLETE)
        END IF
    END FOR
END FUNCTION
```

```
FUNCTION HYBRID_IO_LOOP()
    WHILE (SYSTEM_RUNNING) DO

        // Low-speed path: periodic polling
        FOR EACH device IN low_speed_devices DO
            status = READ_STATUS_REGISTER(device)
            IF (status == DATA_READY) THEN
                data = READ_DATA_REGISTER(device)
                PROCESS(data)
            END IF
        END FOR
        SLEEP(poll_interval)   // bounded delay between polls

        // High-speed path: event-driven, handled outside this loop
        // (see interrupt handler below; no CPU polling for these devices)
    END WHILE
END FUNCTION

ON INTERRUPT(device IN high_speed_devices, type = TRANSFER_COMPLETE) DO
    ACKNOWLEDGE_INTERRUPT(device)
    buffer = DMA_Controller.GET_COMPLETED_BUFFER(device)
    PROCESS(buffer)
    DMA_Controller.START_NEXT_TRANSFER(device)   // re-arm for continuous
streaming
END
```

5.3 Explanation

- Devices are classified once at startup using a throughput threshold; low-speed devices are placed in a periodic polling loop, which is acceptable since their infrequent data availability does not justify the overhead of interrupt setup.
- High-speed devices are instead configured with DMA and an interrupt-on-completion, so the CPU never polls them and never touches their bulk data — it is only invoked once per completed transfer via the interrupt handler.
- The high-speed interrupt handler immediately re-arms the DMA transfer (START_NEXT_TRANSFER), enabling continuous streaming without requiring the CPU to reconfigure the transfer from scratch each time.
- This hybrid design balances simplicity (polling) for devices where interrupt/DMA overhead is unjustified, against efficiency (DMA + interrupts) for devices where CPU involvement in bulk transfer would be a bottleneck.

— End of Solutions —